

```

0000 with IO_EXCEPTIONS;
generic
type ELEMENT_TYPE is private;
package SEQUENTIAL_IO is
type FILE_TYPE is limited private;
type FILE_MODE is (IN_FILE, OUT_FILE);
-- File management
procedure CREATE( FILE : in out FILE_TYPE; MODE : in FILE_MODE; NAME : in STRING; FORM : in STRING;
NAME : in STRING; FORM : in STRING;
procedure CLOSE( FILE : in out FILE_TYPE );
procedure DELETE( FILE : in out FILE_TYPE );
procedure RESET( FILE : in out FILE_TYPE; MODE : in FILE_MODE );
procedure RESET( FILE : in out FILE_TYPE );
function MODE( FILE : in FILE_TYPE ) return FILE_MODE;
function NAME( FILE : in FILE_TYPE ) return STRING;
function FORM( FILE : in FILE_TYPE ) return STRING;
function IS_OPEN( FILE : in FILE_TYPE ) return BOOLEAN;
-- Input and output operations
procedure READ( FILE : in FILE_TYPE; ITEM : out ELEMENT_TYPE );
procedure WRITE( FILE : in FILE_TYPE; ITEM : in ELEMENT_TYPE );
function END_OF_FILE( FILE : in FILE_TYPE ) return BOOLEAN;
-- Exceptions
STATUS_ERROR : exception renames IO_EXCEPTIONS.STATUS_ERROR;
MODE_ERROR : exception renames IO_EXCEPTIONS.MODE_ERROR;
NAME_ERROR : exception renames IO_EXCEPTIONS.NAME_ERROR;
USE_ERROR : exception renames IO_EXCEPTIONS.USE_ERROR;
DEVICE_ERROR : exception renames IO_EXCEPTIONS.DEVICE_ERROR;
END_ERROR : exception renames IO_EXCEPTIONS.END_ERROR;
DATA_ERROR : exception renames IO_EXCEPTIONS.DATA_ERROR;
private
type FILE_TYPE is record
is record
null; -- for front end only
end record;
end SEQUENTIAL_IO;

```